SUB-EVENT DETECTION

L.E.S. Is More
Lara Hofman, Elio Samaha, Sandro Mikautadze

Academic Year 2024/25 - École Polytechnique

Structure

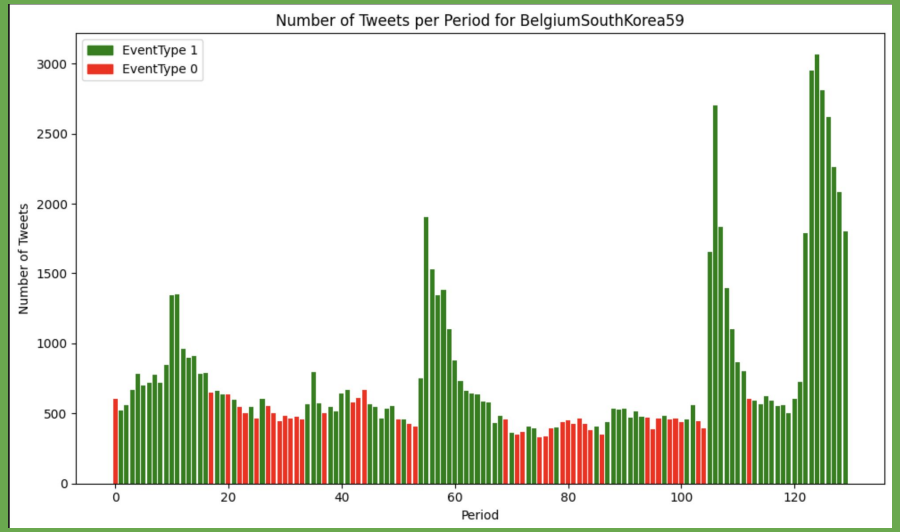
- **Data Processing**
  - Cleaning
  - Feature Engineering
  - Tokenization and Embeddings
- **Models and Results**
  - Simple: Random Forest
  - Complex: LSTM
  - Ensemble: Random Forest + LSTM

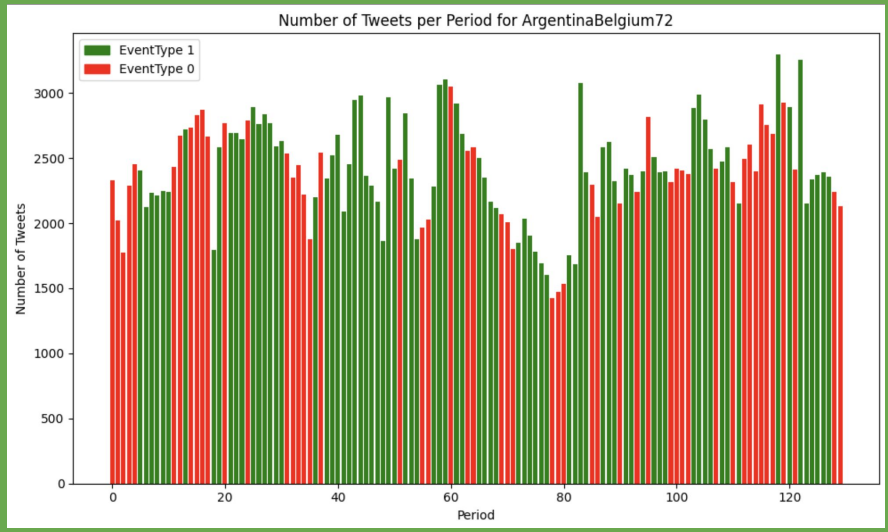
Data Processing

Data Processing: Cleaning

- Removed duplicate tweets and retweets
- Text Normalization:
  - Converted text to lowercase.
  - Replaced:
    - User mentions with @user.
    - URLs with HTTPURL.
    - Emojis with text representations.
    - Team names with <team1>, <team2>, or <team>
- Reduced dataset size significantly (~50% reduction)

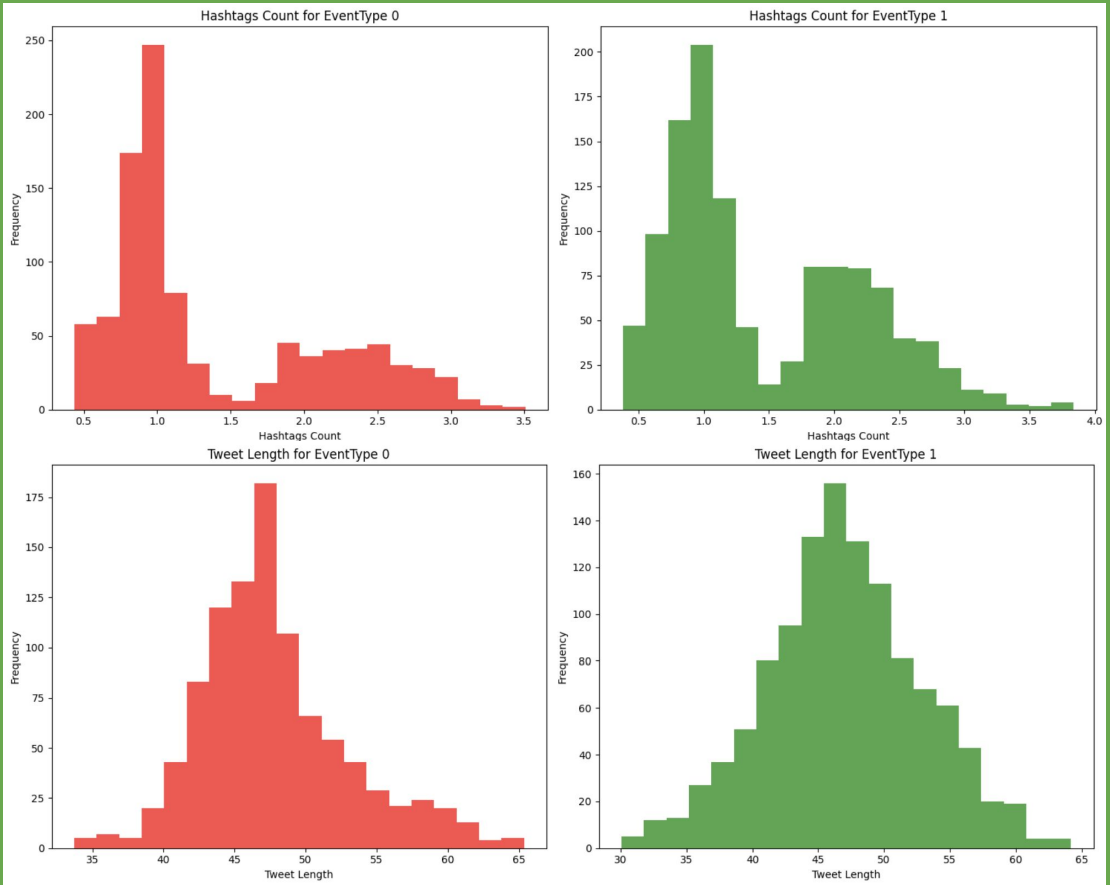
Data Processing: Feature Engineering





- Time dependency
- Team-specificity

Data Processing: Feature Engineering

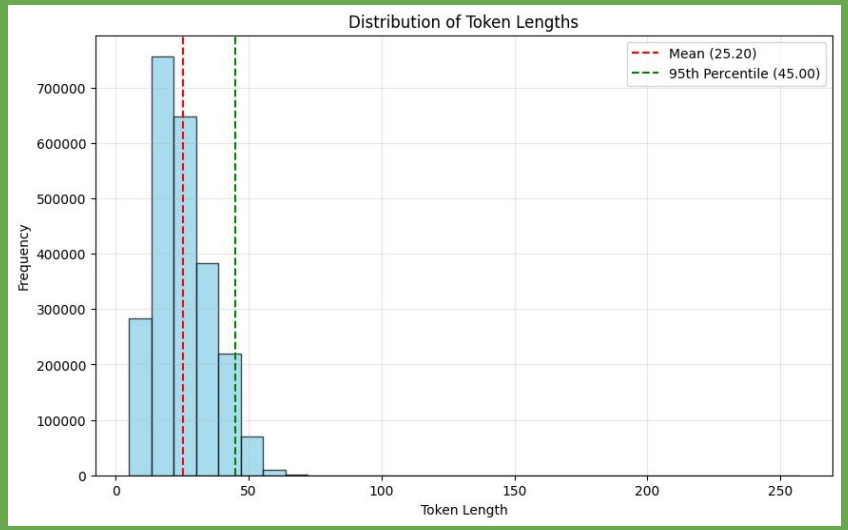


Data Processing: Feature Engineering

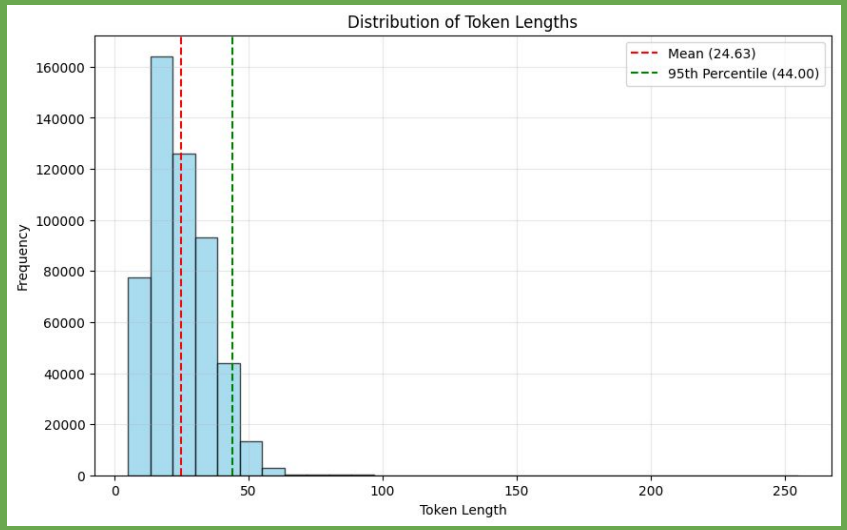
- **Tweet-level Features**:
  - TweetLength: Number of characters in a tweet
  - Exclamation_Count: Number of exclamation marks (!)
  - Hashtags_Count: Number of hashtags per tweet
  - Top-TFIDF Words:
    - Count of words that appeared in the top-30 TF-IDF scores across the corpus per tweet
- **Frequency Features**:
  - TweetCount_PerPeriod: Number of tweets posted in each period.
  - Previous Period Frequencies:
    - Tweet frequencies for the previous 5 periods (e.g., Previous1_TweetFreq, Previous2_TweetFreq, etc.).
- **Team-specific Features**:
  - Team_Score: Absolute difference in performance scores between two teams.

Data Processing: Tokenization

- BERTweet: specifically designed for English tweets
  - same architecture as BERT but with pre-training procedures of RoBERTa
- Tokenization (Byte-Pair Encoding) with maximum length of 50 tokens



Train



Test

Data Processing: Embedding Tweets

- For every token $\mathbf{t_i}$ ($\mathbf{i \in \{1,...,50\}}$), get the embedding $\mathbf{e_i \in R^{768}}$
  - thus getting $\mathbf{E = [e_1,e_2,...,e_{50}] \in R^{768x50}}$ for every tweet
- For every tweet, just keep $\mathbf{e_1}$
  - thus keeping just the embedding of the CLS token

So now every tweet has a corresponding 768-dimensional vector that represents the whole sentence. **How can we get an embedding for each time period?**

ID	Some representation	EventType
0	*embedding_0*	0
1	*embedding_1*	1
…	….	…
2137	*embedding_2137*	0

Data Processing: Aggregating Embeddings by…

- Similarity

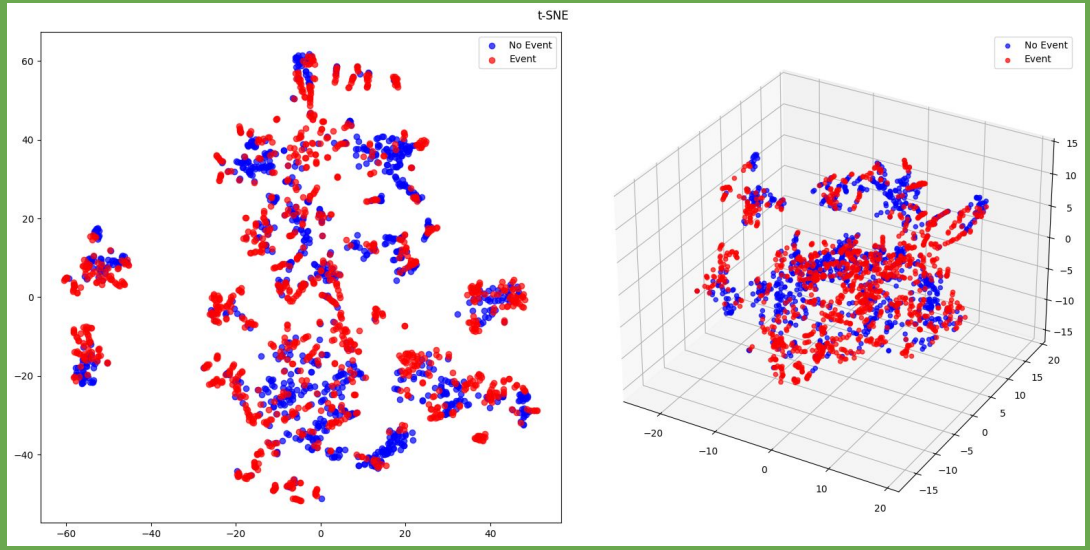
For an *ID* consider the embeddings $\mathbf{E_k = [e_1,...,e_n]}$ for each *EventType* $\mathbf{k \in \{0,1\}}$, where $\mathbf{e_i}$ is the CLS embedding of a tweet, and **n** is the number of tweets in the *ID*

1. Compute mean reference embedding $\mathbf{r_k} = \frac{1}{n}\sum_{i=1}^{n} e_i$ for each **k**
2. Compute cosine similarity $\mathbf{sim(e_i,r_k)} = \frac{e_i \cdot r_k}{||e_i||||r_k||}$ for each $\mathbf{e_i \in E_k}$ **(i = 1,...,n)**
3. Normalize the values $\mathbf{w_i} = \frac{\exp(sim(e_i, r_k))}{\sum_{l=1}^{n} \exp(sim(e_l, r_k))}$ **(i = 1,...,n)**
4. Compute aggregation $\mathbf{E_k^{agg}} = \sum_{i=1}^{n} w_i e_i$ for each **k**

Data Processing: Aggregating Embeddings by…

- Similarity

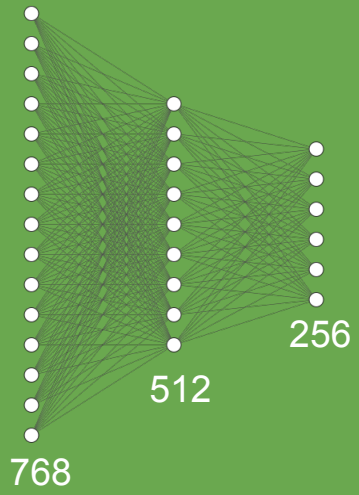
5. Get the aggregated embedding of the ID as $\mathbf{E}^{final} = \frac{|E_0|}{|E_0| + |E_1|} \mathbf{E}_0^{agg} + \frac{|E_1|}{|E_0| + |E_1|} \mathbf{E}_1^{agg}$



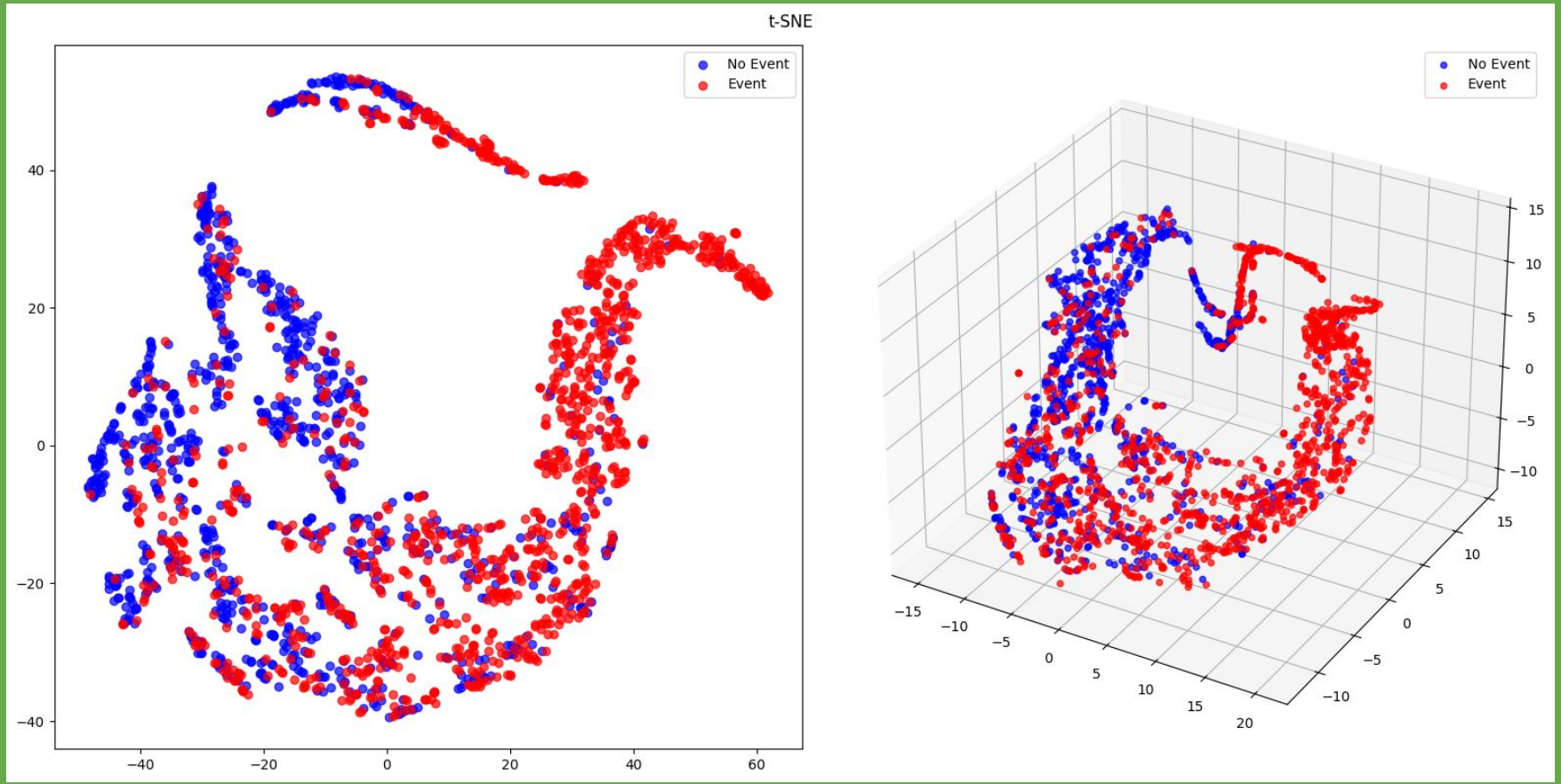
Data Processing: Aggregating Embeddings by…

- Similarity + **Contrastive Learning**

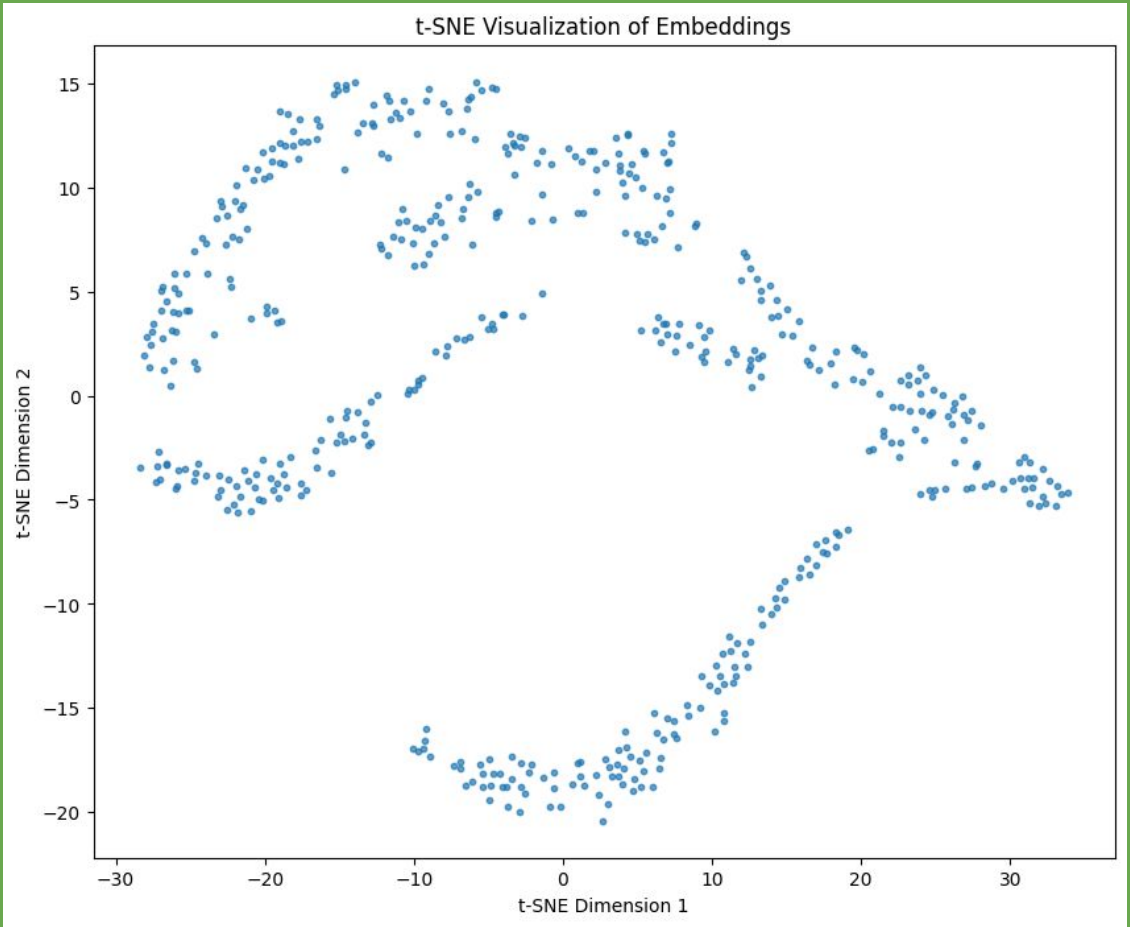
6. Project $\mathbf{E^{final} \in R^{768}}$ into a 256-dimensional vector using an MLP trained with supervised contrastive loss. Let **N=2137** and **i** be the *ID* (so $\mathbf{E_i^{final} \in R^{768}}$ is the embedding of each ID) and **P(i)** the set of IDs with same label as **i**



$$\mathcal{L}_{\text{contrastive}} = -\frac{1}{N}\sum_{i=1}^{N}\frac{1}{|P(i)|}\sum_{p\in P(i)}\log\frac{\exp(E_i^{final}\cdot E_p^{final}/\tau)}{\sum_{j=1}^{N}\exp(E_i^{final}\cdot E_j^{final}/\tau)}$$



Similarity Aggregation + Contrastive Learning



Does it generalize well on test?

Models and Results

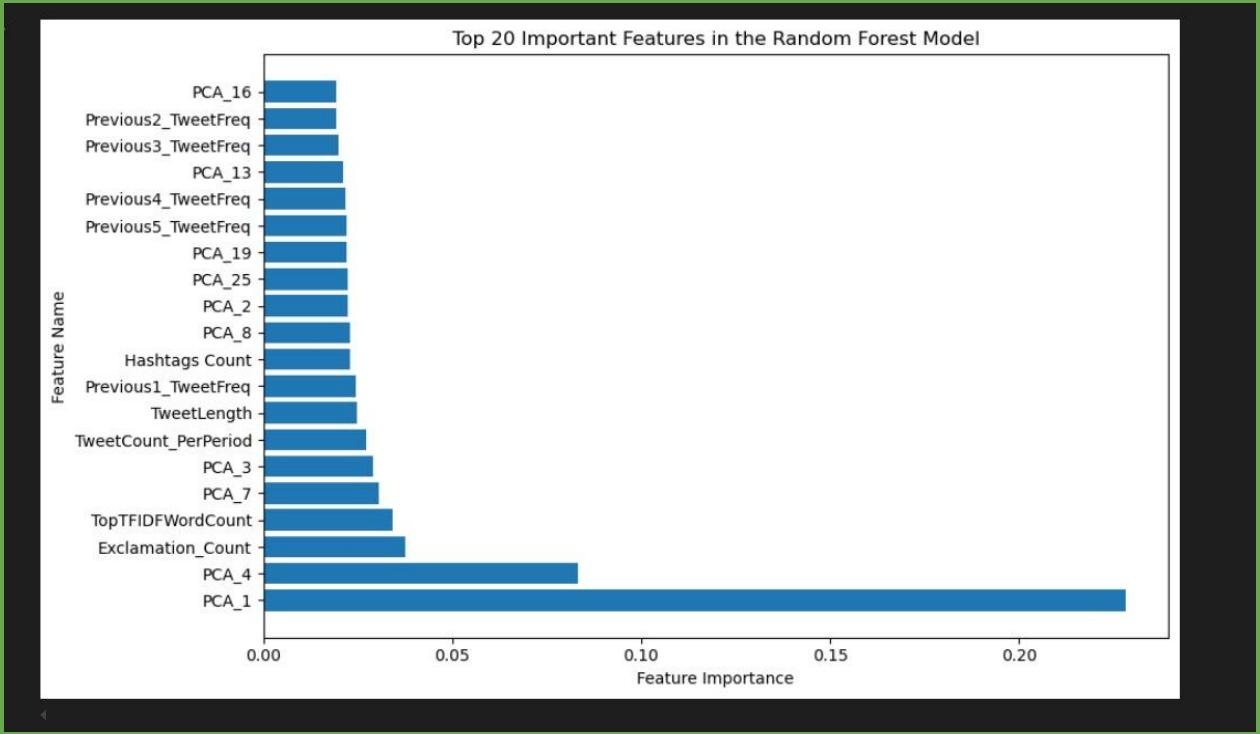
Models: Random Forest - Basic Model

- Preprocessed data with GloVe embeddings (200 dimensions)
- Hyperparameter fine-tuning using Grid Search
  - n_estimators: 50
  - max_depth: None
  - min_samples_split: 2
- Leaderboard performance:
  - public: **0.66015**
  - private: **0.53481**

Models: Random Forest - Complete Model

- Combined engineered features and PCA-reduced embeddings:
  - PCA-reduced (our) embeddings: Reduced to 25 dimensions (99% variance retained)
  - Total Input Features: 37 features (25 PCA dimensions + 12 additional features)
- Hyperparameter Optimization using Grid Search: 'max_depth': None, 'min_samples_split': 10, 'n_estimators': 200
- Cross-Validation

Feature Importance: Random Forest

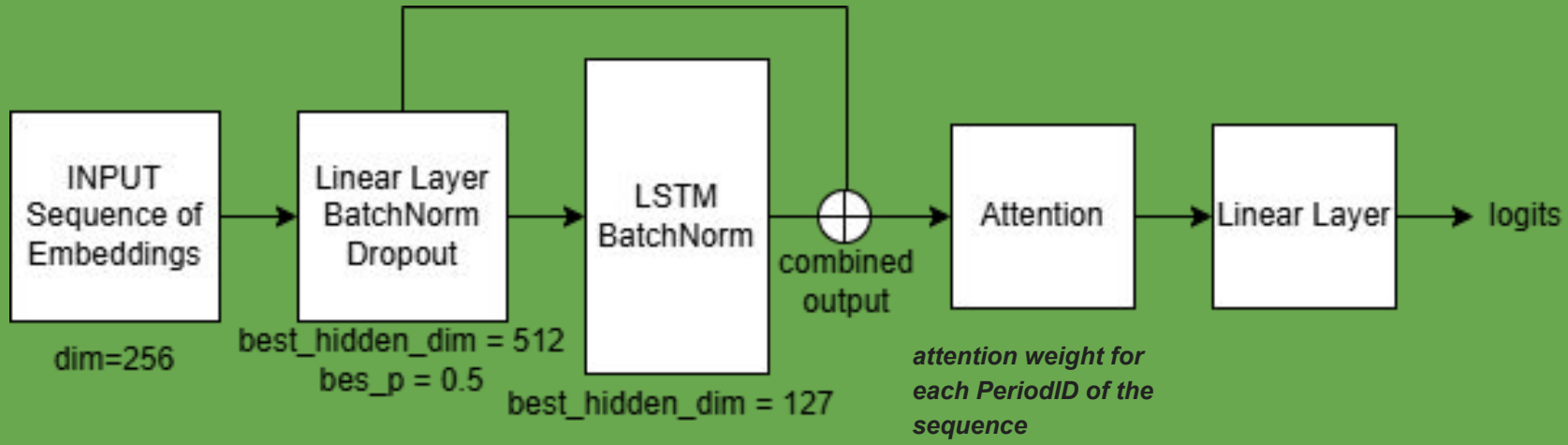


Results: Random Forest - Complete Model

Our chosen model for the submission

- Leaderboard performance:
  - public: **0.76562 (4th)**
  - private: **0.61923 (79th)**
- Reasons for failure:
  - Overfitting (max_depth = None)
  - Insufficient Regularization
    - The balance between depth, splits, and generalization was not optimal
  - Feature and embedding complexity
  - PCA maybe too simplistic and remove important info from the embeddings

Models: LSTM


INPUT
Sequence of
Embeddings
dim=256
Linear Layer
BatchNorm
Dropout
best_hidden_dim = 512
bes_p = 0.5
LSTM
BatchNorm
best_hidden_dim = 127
combined
output
Attention
attention weight for
each PeriodID of the
sequence
Linear Layer
logits

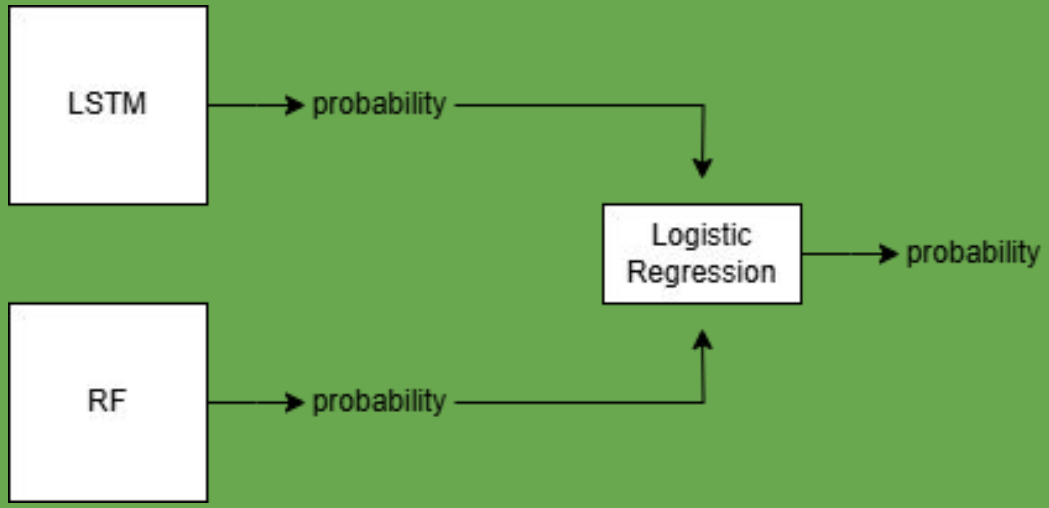
Results: LSTM

- Mean-cross validation accuracy: 0.7721 across folds
- Leaderboard performance:
  - public: 0.72265
  - private: 0.71538
- Stable
- **But can we do more?**

Models: Ensemble (Stacking)

- LSTM is good with sequences and more stable
- RF can handle well tabular data (macro features)

So we combine the two probabilities of each model using a logistic regression



Results: Ensemble

- Mean-cross validation accuracy: 0.7889 across folds
- Leaderboard performance:
  - public: 0.57812
  - private: 0.52692
- Overfitting
- Seems promising and can be improved by further exploration

Thank you.